Embedded Real-Time Control, Alberto Morato

Laboratory 2, Motor Parameters Identification

Last update: NO-Version 10 novembre 2026

1 Introduction

Purpose of this laboratory is to identify the first-order model of the DC motors mounted on the TBot. The identification will be performed by controlling in open-loop the motors, by applying a known step voltage input and measuring the angular speed response. The collected data will be used to estimate the motor parameters.

2 Preliminary Information

2.1 Data logger

The TBot is equipped with a data logger that can record almost any type of data. There are two available communication interfaces:

- Serial Datalogger (via USB debug interface)
- WiFi Datalogger

2.1.1 MATLAB

The serial datalogger is provided as a MATLAB app. On your own computer, extract the archive available on Moodle, open the `DataLoggerApp` folder containing `setup.m`, and run:

```
setup
```

On laboratory computers, use the shared installation instead: run the supplied `dataloggerSetup.m` script. Repeat the appropriate setup command in each new MATLAB session, since the scripts do not permanently save the MATLAB path.

Connect the TBot through its USB debug interface. Use `serialportlist("available")` to list available serial ports and identify the board's port; do not assume a fixed COM number. Close any serial monitor using that port.

Open the application with:

```
serialDataLoggerApp
```

Opening the app does not start acquisition. Configure the following tabs before setting the connection switch to **On**:

- **Connection**: select the board's **Serial Port** and set **Baudrate** to the value used by the firmware (115200 in the example below). For Lab 2, the **Send Transmission Start/Stop Commands** checkbox **must be checked**. Enter the byte values expected by the firmware. The app defaults are 65 and 66 (ASCII A and B); verify these against the supplied STM32 project.
- **Packet**: describe the transmitted fields in their exact order using **Signal Type** and **Dimension**. For the STM32 structure shown below, use two rows of type `single`, each with dimension 2: the first contains `w1, w2`, and the second contains `u1, u2`. Use the table's context menu to add or reorder rows. Set **COBS Encoded** and **Null terminated** to match the firmware; both are enabled by default. The app expects binary numeric data in little-endian order, without padding between fields. Zero termination requires COBS encoding.
- **Log**: set **Tx Sample Time [s]** to the actual interval between transmitted packets, as determined by the STM32 timer and any transmission decimation. This setting reconstructs the time axis; it does not configure the STM32 timer. Set **Buffer size** large enough for the whole experiment: for duration D and packet period T_s, allow at least $\lceil D/T_s \rceil$ samples plus a margin. Once full, the buffer retains the newest samples and replaces the oldest ones. Enable **Log to Workspace** and choose a **Variable name**, for example `data`.
- **Plots**: choose the plot grid and select the signal components to display in the plot tree. Configure the displayed duration in **X Axis**, and automatic or fixed limits in **Y Axis**. **Refresh every # packets received** controls display updates without subsampling the exported data.

For a repeatable configuration, the same settings can be supplied when opening the app. The following example expects four single-precision values grouped as two vector signals and displays angular speeds in the first plot and voltages in the second. Replace `COM8` and the illustrative 0.01 s packet period with the values used in your experiment:

```
app = serialDataLoggerApp( ...
    serialPortName="COM8", baudRate=115200, ...
    enableStartStopCmds=true, ...
    packetSpec={'2*single','2*single'}, ...
    enableCOBS=true, enableNullTerminator=true, ...
    txSampleTime=0.01, bufferSize=100000, ...
    enableLogToWorkspace=true, enableSaveOnExit=true, ...
```

```
    varName="data", ...
    plotsGridSize=[2 1], ...
    plotsContentList=[1 1 1; 1 1 2; 2 2 1; 2 2 2], ...
    plotsDuration=5, plotsRefreshRatio=10);
```

Each entry of `packetSpec` describes one signal; `'2*single'` means two components of type `single`. Each row of `plotsContentList` is `[plotNumber signalNumber componentNumber]`, with indices starting at 1. For Lab 2, keep `enableStartStopCmds=true`, as shown above. Set `txStartCmd` and `txStopCmd` to the byte values expected by the firmware, or configure them in the app.

Set the switch to **On** to start the connection. The green **Connected** indicator confirms that the port is open; also verify that samples arrive. At the end of the trial, set the switch to **Off**, then click **Save to workspace**. Connection and packet settings can only be changed while disconnected.

The exported variable `data` is a structure with these fields:

- `data.time`: reconstructed sample times in seconds. Time increases by the configured T_s for each accepted packet, starting at T_s for the first sample. These are not measured arrival times and do not reveal missing packets.
- `data.signals`: a cell array with one element per signal in `packetSpec`. Each matrix has components on rows and samples on columns. For example, `data.signals{2}(2,:)` retrieves `u2` for the packet configuration above.
- `data.sampleIndex`: indices of the retained samples.
- `data.size`: number of exported samples.

Save to workspace exports the samples currently retained in the buffer and overwrites any variable with the same name. It does not create a file on disk. **Save to workspace on exit** also exports when the window closes, provided **Log to Workspace** is enabled. Save each trial to a MAT file before starting another one; **Clear buffer** discards the buffered samples.

For the packet layout above, retrieve and save the motor measurements with:

```
% After clicking Save to workspace:
assert(data.size > 0, 'No samples acquired');
t = data.time - data.time(1);
w1 = data.signals{1}(1,:);
w2 = data.signals{1}(2,:);
u1 = data.signals{2}(1,:);
u2 = data.signals{2}(2,:);
save('motor_trial01.mat', 'data');
```

To use the WiFi datalogger, the following command should be invoked:

```
data = udp_datalog(ip, port, packet_specification)
```

where `ip` is the IP address of the robot, `port` is the port number used by the WiFi datalogger, and `packet_specification` is a cell array of char arrays that specifies the type of data to be recorded.

For example, you can invoke the command:

```
data = udp_datalog('147.162.118.61',9090, {'2*single','2*single'}, 'buffersize', 100000)
```



ATTENTION: every turtlebot has a different IP address. Please check (or ask for it) the IP address of the robot you are trying to connect to.

If the serial connection fails, set the switch to **Off**, close the app and any serial monitor, check the USB connection and selected port, then reopen the app. If the port opens but no samples arrive, check the firmware's start command, packet layout, COBS encoding, and termination settings. If the app command is not recognised, rerun the appropriate setup script.

2.1.2 STM32

On the STM32 side, the data logger must be properly configured. Example code is provided in the project for Lab 2.

You can choose which communication interface you want to use by calling the following instructions inside the main function (these function calls are already included in the project):

```
//logger.uart_handle = huart3; // for serial
logger.uart_handle = huart2; // for wifi
```

The basic steps to initialize and run the data logger are explained in the code below:

```
/* declare an ertc_dlog struct */
struct ertc_dlog logger;
.
.
.
int main(void)
{
    .
    .
    .
    /* USER CODE BEGIN 2 */
    /* select the correct UART: uart3 is the default peripheral used */
    logger.uart_handle = huart3;
    // logger.uart_handle = huart2; // for wifi

    /* in this example, we use the timer to periodically send data through the data logger */
    HAL_TIM_Base_Start_IT(&htim6);
    /* USER CODE END 2 */
    .
    .
    .
    while (1)
    {
        /* USER CODE BEGIN 3 */
        /* continuously update the state of the data logger */
        ertc_dlog_update(&logger);
    }
    /* USER CODE END 3 */
}
```

To send arbitrary data, it is necessary to define a struct as follows:

```
struct datalog
{
    float w1, w2;
    float u1, u2;
} data;
```

The structure above is just an example; the structure can be arbitrary, but it is important to put the `datalog` attribute after the word `struct`.

To send the data:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        data.w1 = 10;
        data.w2 += 1.085;
        data.u1 = -3.14;
        data.u2 = 0.555683;
        ertc_dlog_send(&logger, &data, sizeof(data));
    }
}
```

To avoid the case in which the board starts sending "useful" data before MATLAB is ready to show them, it is suggested to check for the value of `tx_enabled` before setting the value of the reference signal. `tx_enabled` indicates whenever the data-logger, from the PC side, is able to receive data.

 **WARNING**: To avoid excessive transmission time and delays please send only numeric data (e.g., float, int, uint8_t, etc.). Do **not** send strings.

3 Exercises

3.1 Exercise 1

- The main goal for this laboratory is to identify the first-order model of the DC motors mounted on the TBot.
- This is done by applying a step voltage input to the motors and measuring the angular speed response.
- According to the **Motor Control lecture**, you need to implement:
  - All the necessary functions to read and compute the angular speed of the motors. (Examples are provided in the Motor Control lecture document.)
  - All the necessary functions to set the voltage applied to the motors using PWM. (Examples are provided in the Motor Control lecture document.)
  - All the steps needed to accomplish the goal should be executed from inside the `HAL_TIM_PeriodElapsedCallback` function, which, as previously mentioned, is a callback function called when a timer's period is elapsed. Remember to check which timer caused the callback to be called: in particular, we are interested in handling the events raised by the timer TIM6.
- According to the **Motor Parameters identification lecture**, acquire the step response of **both the motors** by applying a step voltage input and measuring the angular speed response.
  - Selection of the step amplitude is left to the student. The choice must be motivated. Is suggested to acquire multiple step responses with different amplitudes.
  - Selection of the control mode (e.g., Forward/Brake or Forward/Coast) is left to the student. You can select one or try both. The choice must be motivated.
  -  **WARNING**: The maximum voltage for the motor is 6V. Supposing a 12V power supply, the maximum duty cycle for the PWM signal **needs to be limited to 50%**.
- With the collected data, estimate the motor parameters using the **tangent method** and one of the other two methods presented in the Motor Parameters identification lecture.
  - Compare the results obtained with the different methods.
- According to the identified first-order model, find the values of the k_p and k_i constants for a PI controller that regulates the angular speed of the motors.
  - The technique to find the k_p and k_i constants is left to the student. You can use any method presented in the lectures or any other method you prefer. The choice must be motivated.
- Analyze in Matlab the closed-loop step response of the system with the designed PI controller.
- **The found k_p and k_i constants will be used in next laboratory sessions to implement the PI controller on the TBot.**

3.2 Hint: start the motors when the logger is active

In the supplied code, the flag mentioned above is named `logger.tx_enable`. The function `ertc_dlog_update(&logger)`, called continuously in the main loop, sets it to true when it receives the Start command from MATLAB (byte 65) and to false when it receives Stop (byte 66). In the MATLAB app, enable **Send Transmission Start/Stop Commands**: switching the connection **On** sends Start after preparing reception; switching it **Off** sends Stop.

For example, inside the TIM6 callback, before your existing code that converts motor voltages to PWM, choose the voltages as follows:

```
if (logger.tx_enable) {
    data.u1 = 3.0f;  // test voltage for motor 1 [V]
    data.u2 = 3.0f;  // test voltage for motor 2 [V]
} else {
    data.u1 = 0.0f;
    data.u2 = 0.0f;
}

/* Your code: apply data.u1 and data.u2 using PWM,
   read the motor speeds into data.w1 and data.w2,
   then send the packet as in the previous example. */
```

This way, the motors receive the test voltage only when logging is enabled. Your PWM code must also apply the zero-voltage commands when the flag is false. Keep the timer running and keep calling `ertc_dlog_update` in the main loop. The send function already checks the same flag. Record the applied voltages together with the speeds to identify the step in the log. The flag reflects received commands: unplugging the cable does not automatically reset it.